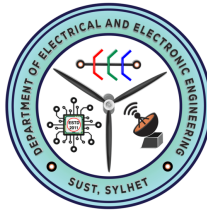


Lab Report

Experiment: 7

Design and Simulation of a D Flip-Flop with Enable Using a 2:1 Multiplexer in Verilog HDL.



Department of Electrical and Electronic Engineering

Course Title: VLSI Design and Testing Lab

Course Code: EEE 456

Submitted By:

Prapto Debnath (Reg: 2021338016)

Sujon Bepary (Reg: 2021338044)

Group No : 12

Submitted To:

Myen Uddin

Lecturer

Department of Electrical & Electronic Engineering

Shahjalal University of Science and Technology, Sylhet

Contents

1	Introduction	1
2	Theoretical Background	1
2.1	D Flip-Flop	1
2.2	2:1 Multiplexer	1
2.3	Enable Function	2
3	Circuit Design	2
4	Necessary Verilog Codes	3
4.1	dff.v	3
4.2	mux2to1.v	4
4.3	main_mod.v	4
4.4	main_mod_tb.v	5
5	Test Cases	6
6	Simulation Results	7
6.1	Reset Operation	8
6.2	Hold Operation	8
6.3	Load Operation	8
7	Result Analysis	8
8	Discussion	9
9	Conclusion	10

1. Introduction

Sequential logic circuits are fundamental building blocks of digital systems because they can store information and maintain a state over time. A D flip-flop is one of the simplest and most widely used sequential storage elements. In practical digital systems, it is often necessary to update a stored value only when an enable signal is asserted. An enable-controlled D flip-flop provides this functionality.

In this experiment, the enable function is implemented structurally using a 2:1 multiplexer placed before a D flip-flop. One multiplexer input is connected to the external data signal, while the other is connected to the present output of the flip-flop. When the enable signal is HIGH, new data is selected and loaded at the next rising edge of the clock. When the enable signal is LOW, the feedback path is selected and the flip-flop retains its previous state.

The design is described using Verilog HDL and verified through behavioral simulation in ModelSim.

2. Theoretical Background

2.1. D Flip-Flop

A D flip-flop stores one bit of information and changes its output at the active edge of the clock. For a positive-edge-triggered D flip-flop,

$$Q_{next} = D$$

at the rising edge of the clock.

The D flip-flop used in this experiment also has an asynchronous active-low reset. Thus,

$$RESET = 0 \Rightarrow Q = 0.$$

Since the reset is asynchronous, Q can be cleared without waiting for a clock edge.

2.2. 2:1 Multiplexer

A 2:1 multiplexer selects one of two input signals according to a select signal. For the multiplexer used here,

$$Y = \begin{cases} S_0, & SEL = 0, \\ S_1, & SEL = 1. \end{cases}$$

Equivalently,

$$Y = \overline{SEL}S_0 + SELS_1.$$

2.3. Enable Function

The MUX is connected so that:

- $S_0 = Q$, the present flip-flop output.
- $S_1 = DATA$, the external data input.
- $SEL = EN$, the enable signal.

Therefore,

$$D_{FF} = \begin{cases} Q, & EN = 0, \\ DATA, & EN = 1. \end{cases}$$

The resulting next-state equation is

$$Q_{next} = EN \cdot DATA + \overline{EN} \cdot Q$$

when reset is inactive.

Including the asynchronous reset,

$$Q_{next} = \begin{cases} 0, & RESET = 0, \\ DATA, & RESET = 1, EN = 1, \\ Q, & RESET = 1, EN = 0. \end{cases}$$

3. Circuit Design

The circuit consists of two main modules: a 2:1 multiplexer and a D flip-flop. The output of the flip-flop is fed back to one input of the multiplexer. The external data is connected

to the other MUX input.

The feedback path causes the circuit to hold its previous state when the enable input is LOW. When enable is HIGH, the external data is selected and captured on the next rising clock edge.

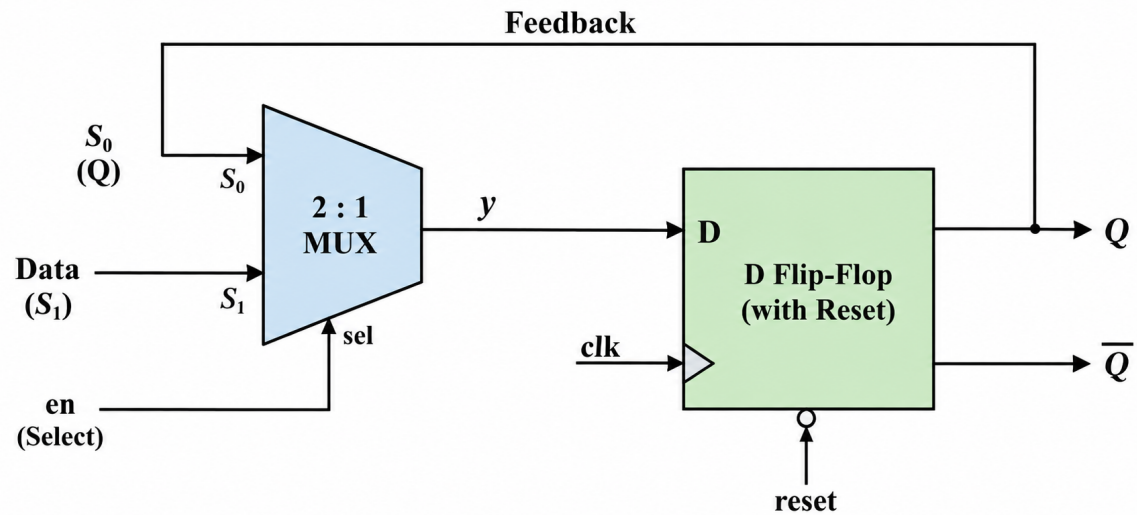


Figure 1: Functional Block diagram of the enabled D Flip-Flop.

4. Necessary Verilog Codes

4.1. dff.v

```
1 module dff(  
2     input wire clk,  
3     input wire reset,  
4     input wire d,  
5     output reg q  
6 );  
7  
8     always @(posedge clk or negedge reset)  
9         if (~reset)  
10            q <= 1'b0;  
11        else  
12            q <= d;  
13  
14 endmodule
```

Listing 1: Verilog implementation of dff.v

4.2. mux2to1.v

```
1 module mux2to1(  
2     input wire s0,  
3     input wire s1,  
4     input wire sel,  
5     output reg y  
6 );  
7  
8     always @(*)  
9         if (sel)  
10            y = s1;  
11        else  
12            y = s0;  
13  
14 endmodule
```

Listing 2: Verilog implementation of mux2to1.v

4.3. main_mod.v

```
1 module main_mod(  
2     input wire data,  
3     input wire en,  
4     input wire clk,  
5     input wire reset,  
6     output wire q  
7 );  
8  
9     wire mux_out;  
10  
11     mux2to1 mux1(  
12         .s0(q),  
13         .s1(data),  
14         .sel(en),  
15         .y(mux_out)  
16     );  
17  
18     dff dff_1(  
19         .d(mux_out),  
20         .reset(reset),
```

```

21     .clk(clk),
22     .q(q)
23 );
24
25 endmodule

```

Listing 3: Verilog implementation of main_mod.v

4.4. main_mod_tb.v

```

1  `timescale 1ns/1ps
2
3  module main_mod_tb;
4
5      reg clk;
6      reg reset;
7      reg data_1;
8      reg en_1;
9
10     wire q;
11
12     main_mod DUT(
13         .data(data_1),
14         .clk(clk),
15         .reset(reset),
16         .en(en_1),
17         .q(q)
18     );
19
20     initial begin
21         clk = 0;
22         forever #5 clk = ~clk;
23     end
24
25     initial begin
26         reset  = 0;
27         en_1   = 0;
28         data_1 = 0;
29         #10;
30
31         reset  = 1;

```

```
32     en_1    = 0;
33     data_1 = 1;
34     #10;
35
36     en_1    = 1;
37     data_1 = 0;
38     #10;
39
40     data_1 = 1;
41     #10;
42
43     en_1    = 0;
44     data_1 = 0;
45     #10;
46
47     data_1 = 1;
48     #10;
49
50     en_1    = 1;
51     data_1 = 0;
52     #10;
53
54     data_1 = 1;
55     #10;
56
57     $stop;
58 end
59
60 endmodule
```

Listing 4: Verilog testbench `main_mod_tb.v`

5. Test Cases

The testbench was designed to verify reset, hold, and load operations.

Table 1: Applied test conditions and expected behavior

Test	RESET	EN	DATA	Expected Q
1	0	0	0	0
2	1	0	1	Hold at 0
3	1	1	0	0
4	1	1	1	1
5	1	0	0	Hold at 1
6	1	0	1	Hold at 1
7	1	1	0	0
8	1	1	1	1

6. Simulation Results

The final waveform contains the signals `clk`, `reset`, `data_1`, `en_1`, and `q`. The observed output follows the expected behavior of the enable-controlled D flip-flop.

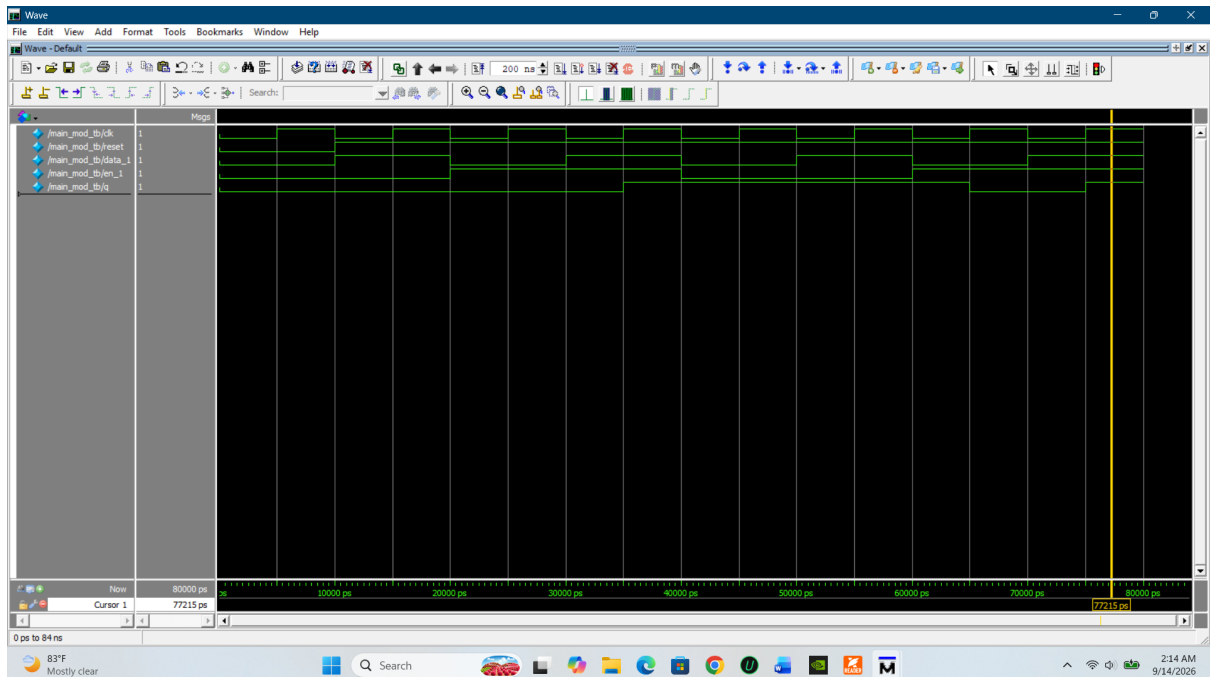


Figure 2: ModelSim waveform of the simulated enable-controlled D flip-flop.

6.1. Reset Operation

Initially, the reset signal is LOW. Because the DFF has an asynchronous active-low reset, the output Q is forced to logic 0. This occurs independently of the clock.

Therefore,

$$RESET = 0 \Rightarrow Q = 0.$$

6.2. Hold Operation

When the reset is released and

$$EN = 0,$$

the MUX selects its S_0 input, which is the feedback signal Q. Thus, the D input of the flip-flop becomes equal to its current output:

$$D_{FF} = Q.$$

Consequently, the output remains unchanged even if DATA changes.

6.3. Load Operation

When

$$EN = 1,$$

the MUX selects DATA:

$$D_{FF} = DATA.$$

At the next rising edge of the clock, the DFF captures the selected data. Thus, if DATA is 0, Q becomes 0, and if DATA is 1, Q becomes 1.

7. Result Analysis

The simulation verifies all required functional conditions.

Table 2: Observed functional behavior

Condition	Observed behavior
RESET = 0	Q is cleared to 0 asynchronously.
RESET = 1, EN = 0	Q retains its previous stored value.
RESET = 1, EN = 1, DATA = 0	Q becomes 0 at the next rising clock edge.
RESET = 1, EN = 1, DATA = 1	Q becomes 1 at the next rising clock edge.

The waveform therefore agrees with the theoretical next-state relation

$$Q_{next} = EN \cdot DATA + \overline{EN} \cdot Q$$

when reset is inactive.

The experiment also confirms that the MUX-based implementation correctly reproduces the behavior of a D flip-flop with enable.

8. Discussion

The experiment demonstrates how a simple combinational circuit can be combined with a sequential circuit to implement a useful storage function. The 2:1 multiplexer performs the selection required for the enable operation, while the D flip-flop provides the actual state storage.

The feedback connection from Q to the MUX is the key to the hold function. When EN is LOW, the current state is selected and returned to the D input. At the next rising clock edge, the flip-flop therefore stores the same value it already contains.

When EN is HIGH, the feedback path is bypassed and the external DATA signal is selected. The value is captured at the next positive edge of the clock. This behavior is consistent with the standard enable-controlled D flip-flop equation.

The asynchronous reset provides a reliable initial state. Since the reset is included in the sensitivity list as `negedge reset`, asserting reset LOW forces Q to zero immediately rather than waiting for a clock transition.

The successful ModelSim compilation and waveform simulation show that the structural interconnection of the modules is correct. The simulation results also demonstrate the distinction between combinational selection in the MUX and edge-triggered storage in the DFF.

9. Conclusion

An enable-controlled D flip-flop was successfully designed using a 2:1 multiplexer and a positive-edge-triggered D flip-flop with an asynchronous active-low reset. The complete design was described in Verilog HDL using separate modules for the DFF, multiplexer, top-level circuit, and testbench.

The design was compiled and simulated successfully in ModelSim. The simulation waveform verified the expected reset, load, and hold operations. When the enable signal is HIGH, the circuit captures the external data at the rising clock edge; when the enable signal is LOW, the previous output is retained.

Thus, the implemented circuit satisfies

$$Q_{next} = \begin{cases} 0, & RESET = 0, \\ DATA, & RESET = 1, EN = 1, \\ Q, & RESET = 1, EN = 0. \end{cases}$$

The experiment successfully demonstrates both the Verilog HDL implementation of sequential logic and its functional verification using ModelSim.